

CPRE 4310

M03 HW

Assignments will be submitted in PDF format via Canvas.

Please submit your homework online through Canvas. Late homework will not be accepted.

Important: Your submission must be in .pdf format ONLY!

Please ensure that you support all your answers with the correct screenshots showing your solutions.

You are not allowed to use AI tools to answer any of the HW questions

1. Briefly state the differences between DAC and MAC access control mechanisms.

Discretionary Access Control (DAC) and Mandatory Access Control (MAC) have quite a few differences. One of the most prominent difference between them is that DAC works by controlling access based on the identity of the requester and on the basis of access rules (to determine what users can and cannot do). Whereas MAC as the name suggests, controls access to resources based on comparing security labels with security clearances instead.

Additionally, MAC is mostly used in government organizations where high security is needed for sensitive data. DAC is more flexible as compared to MAC so it is used in organizations where there is a need for more user-based and collaborative type environment.

2. The inclusion of the salt in the UNIX password scheme increases the difficulty of guessing by a factor of 4096. But the salt is stored in plaintext in the same entry as the corresponding ciphertext password. Therefore, salt is known to the attacker. Based on that, why is it asserted that the salt increases security?

Adding a salt in the UNIX password scheme increases the difficulty of guessing by a factor of 4096 because even though the attacker knows the salt, he still needs to use the salt added with different passwords to try to find the correct password. The salt does not increase the passwords security inherently, but instead just makes it harder for the attacker to use rainbow tables to try to find the right password.

To further explain this a little more, basically adding a salt makes pre-computed tables (Rainbow tables) useless. This is because if an attacker gets a hash of a password without a salt, he can check that hash against his Rainbow table to find out the original password. However, if he gets the hash of a password that was stored with a salt, even if he knows the salt, he needs to look at each (pass+salt) combination since every salt is different for each user, and so are their passwords. This makes using Rainbow Tables useless because with every user, there is a different salt attached.

3. In the traditional UNIX file access model, UNIX systems provide a default setting for newly created files and directories, which the owner may later change. The default is typically full access for the owner combined with one of the following: no access for group and other, read/execute access for group and none for other, or read/execute access for both group and other. Briefly discuss the advantages and disadvantages of each of these cases, including an example of a type of organization where each would be appropriate.

One thing to note between all cases is that they all do not allow anyone other than the owner to have write permissions, this is done as a security measure because giving write permissions is as close as giving delete permissions and is often quite dangerous if the file/directory is important.

Case 1: `rwX --- ---`:

The biggest advantage this use case has is that it is the most secure out of all the cases since it only allows only the owner to read, write, and execute a file/directory. However, the main disadvantage that comes with this use case is that it makes it difficult for other users to even see/read the file contents which in turn makes it difficult for collaboration to take place.

Use case: Government Organizations/Agency where security of files is crucial and confidentiality is of utmost importance.

Case 2: `rwX r-X ---`:

This case allows the owner to have read, write, and execute permissions while giving read and execute permissions to groups, and no permissions to others. This use case is overall the most well balanced and practical use case out of the three, mainly due to allowing collaborations to occur with a group but also restricting other users from seeing or executing the file. However, the biggest disadvantage in this use case is that there needs to be someone to constantly watch over the permissions to ensure that the right users are in the right groups. While the file may only be read and executed by group members, there needs to be someone who has a daily/weekly check to ensure only authorized users exist in the group.

Use case: An organization/company that handles a code base that needs to be collaborated on by multiple people within a group. The team leader could be the owner of the file/directory while the “coders” could be the group that needs to read and execute the files for testing purposes.

Case 3: `rwX r-X r-X`:

This case again allows the owner to have read, write, and execute permissions while giving read and execute permissions to both groups and others. The biggest advantage this use case has compared to case 2 is that it does not need much oversight when it comes to the permissions, since both groups and others are allowed to read and execute, allowing for maximum collaborations without any hiccups. However, this is also its biggest disadvantage in a sense (double edged sword) since any user can read and execute the file/directory, this type of setting of permissions is the worst from a security standpoint.

Use case: University Shared Server (canvas) to share lecture notes and documents needed by students authorized to access the server can see their grades, lecture notes, etc.

4. Provide short answers to the following questions:

a. Why should system administrators remove unnecessary services, applications, and protocols?

Removing unnecessary services, applications, or protocols is a hardening technique to ensure that we only keep what we use on our system. This also helps us reduce the number of ways that an attacker can try to access/get into our system.

b. Where is application and service configuration information stored on Unix and Linux systems?

The application and service config info is usually stored in the /etc directory.

c. How does “chroot jail” used to improve application security? And what are its limitations?

A “chroot jail” simply isolates a process and its children from the rest of the file system. The command ‘chroot’ basically changes the root directory from root to a new, restricted directory. Its limitations are that it is not a complete security solution since a process that is running with root permissions can escape this “chroot jail”. Furthermore, it is not as easy as just starting a process with chroot and you have put it in a jail, it is complex and requires copying all files related/needed by the process into the jail directory.

5. Logging is a very important control for operating system and network security, explain the following:

a. Why is logging important?

Logging is especially important when we want to get to the bottom of why an error or crash in the system occurred. We can look deeper into previous logs that were saved by journald to identify what caused our services to run into issues and what the last few logs were just before a system crash to identify the possible actor of the crash.

Furthermore, since logs are basically saved outputs from services, they are crucial for our system security because they provide a detailed and thorough chronological record of all the events occurred/occurring. This recording of data is important because it is used by Intrusion Detection Systems, Incident Response, and System Monitoring.

b. What are its limitations as a security control?

I feel that the main limitation of logging is that it is unable to do anything in particular to recover the system. While it helps us find the root cause of the issues and crashes on our system services, it does not have any features that allow it to do more than just track everything.

On top of this, another key limitation of logging as a security control is that it is a reactive sort of behavior and not a proactive or preventive measure. Another limitation that I can think of is that logging as we know is critical for identifying important things going on in our system and securing it, if our logs go missing or are attacked/deleted by an actor, then we may not be able to get to the bottom of what went wrong (makes it much more difficult to do cyber forensic analysis too).

c. What are the pros and cons of remote logging?

Remote logging allows us to have an improved version of security and integrity, centralized analysis and monitoring of logs (which in turn allows long term storage since it is remote), and simplified audit checks. All these pros allow us to handle remote logs better to a certain extent, and also with logs being remote, they further make access to them more convenient. However, on the flip side, since these logs are centralized, they also have the downside of being a single point of failure. Additionally, there is also a higher level of complexity and cost involved when it comes to logging in a remote manner.

d. Why is it important to rotate log files (overwrite old log files)?

Apart from the fact that log files fill up and should not exceed a certain size (for reading and viewing purposes), there are also other important factors to consider such as the performance, management and compliance side of things. Performance could slow down if there is just one file that has not been rotated and continues logging all the data, and this would be troublesome if we lose that one log file or it becomes corrupt because then we have no other logs to look into.

Furthermore, having smaller sized log files that rotate often would be safer if you know that the system has issues and crashes frequently. Because then after a system crashes, we may lose access to the some files but not all if we have frequent rotations. In addition to losing the log files, there is also a chance that a file could be corrupted, this would also have a smaller impact if the journal logs are rotated in a timely manner.

6. Consider an automated audit log analysis tool (e.g., swatch). Can you propose some generic rules which could be used to distinguish “suspicious activities” from normal user behavior on a system for some organization, for the following activities:

a. User Authentication

- i. We should have a rule dedicated to watch for multiple failed attempts to log in (possible brute force). This rule can be set up to either look out for a user trying many different passwords on one user account, or a single password being used with multiple usernames.
- ii. We should probably also have another rule to monitor where the login attempts are coming from, if it is a single IP address with multiple attempts to log in (either through different usernames or passwords) then it should be flagged.
- iii. Another rule could also be on the lookout for if the login attempts (for a specific username) are coming from many different IP addresses that are very different. (This could be one of those impossible travel situations)

b. Website Access

- i. A rule should be created if the user is accessing various different links one after another. Especially if there is no correlation between the links. For example, one moment the user is looking at the Careers page, next he is on the /admin page or /login page.
- ii. Following up on the above rule, we could set up another similar rule to watch out for malicious scripting or SQLi attacks if the user tries 1=1 or something

similar on the website URL or in one of the forms that may be present on the website.

- iii. A rule should be made for users who try to log in using the same username through multiple different IP addresses in a short time span.
- iv. Another good rule to add would be to see if there are too many requests to login coming from a single IP address. (Possible Brute force attack)